DS 203 PROJECT

CLASSIFYING VEHICLE STATE WITH CONVENTIONAL ML

RUDRA SHANKER - 24B0737
AJAY TEJA K - 24B0617
PRATHAM BAGDI - 24B0700
VISHESH SHRIVASTAVA - 24B3010

INDEX

1. Problem Statement
2. Our Solution Overview
3. Executive Summary
4. Exploratory Data Analysis
5. Feature Engineering
6. Modeling (Random Forest ,Logistic Regression ,SVM and Gradient Boosting)
7. Precision - Recall and ROC Curves
8. Confusion Matix
9. Experimentation
10. Learnings and Challenges

PROBLEM STATEMENT :

Goal : Build a model to recognize vehicle behavior from a camera mounted at top on a moving vehicle.

Target Classes :

HALTED : Vehicle is stationary

AHEAD : Moving forward (includes gradual turns).

BACKWARDS : Moving in reverse.

SHARP-LEFT / SHARP-RIGHT : Executing significant turns.

Required Deliverables :

Image-wise classification (Image_no →→ State).

Segment-wise classification (Start Frame →→ End Frame →→ State).

Constraint :

Must rely on manual feature extraction (no automated deep learning features) and classical ML models (e.g., Logistic Regression, RF, SVC).

Solution Overview

Data split → Labelling → Class Imbalance

↓

Normalize → Feature generation

↓

Training → Error metrics → Roc curves

↓

Refeature → Testing → Model Selection

EXECUTIVE SUMMARY :

This project involves developing a classical machine learning pipeline to determine the movement state of a vehicle (e.g., HALTED , AHEAD , SHARP-LEFT) from a sequence of images. By utilizing a camera mounted on a moving vehicle ,the system analyzes temporal changes between frames to categorize driving maneuvers.

STEP 1 : Data labeling

- Data labelling was done manually.
- The frames were numbered and corresponding direction label was stored as ‘state’.

STEP 2 : Exploratory Data Analysis

- Validated and cleaned video frame dataset with proper labelling and structure
- Ensured correct temporal ordering and handled missing data
- Used consecutive frames to capture motion patterns
- Prepared a reliable dataset for further analysis and modeling

STEP 3 : Feature Engineering

- Feature engineering focused on extracting meaningful motion-based features from consecutive frames
- We computed frame difference metrics to capture overall motion intensity and directional changes across regions (left-right, top-bottom)
- Optical flow was used to estimate pixel-level motion, from which features such as flow direction, magnitude, and asymmetry were derived
- Additional features like motion variance, entropy, gradient ratios, and curl were used to capture complexity and structural movement patterns
- Together, these features provided a compact and informative representation of motion dynamics, enabling effective model learning

STEP 4 : Model Training and Evaluation

Model Training Overview

- **Objective :** Train a machine learning model to accurately predict target classes from engineered features
- **Input :** Preprocessed and scaled feature vectors
- **Output :** Predicted class labels for each data sample
- **Key Focus :** Capturing underlying patterns, relationships, and structural variations in the data

Training Data Preparation

- Applied feature scaling to ensure uniform contribution of all features
- Addressed class imbalance using SMOTE (Synthetic Minority Oversampling Technique)
- Ensured balanced representation across all classes for fair learning
- Performed a structured train-test split to evaluate model generalization

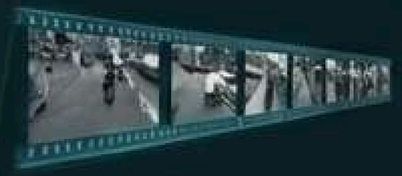
Exploratory Data Analysis :

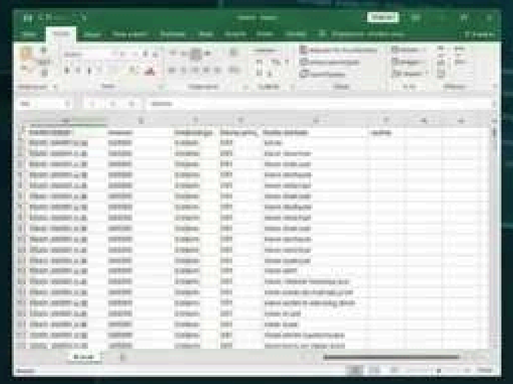
Dataset Overview

- Dataset consists of 21 video sequences
- Each video is stored as frame images (.png)
- Frames organized in folders (0000 → 0020)
- Labels provided via Excel files (train & test)

Data Organization

- Each folder represents one video
- Each Excel sheet corresponds to a video sequence
- Frame names are zero-padded (e.g., 000001.png)
- Ensured correct mapping between Excel & folders





Data Validation

- Verified existence of all folders and files
- Checked sample frames from each video
- Validated image paths using existence checks
- Handled missing/corrupt frames safely

Data Cleaning

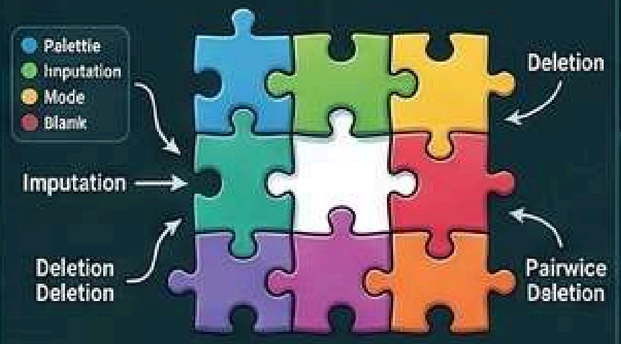
- Standardized column names (lowercase, trimmed)
- Cleaned label values (removed extra spaces)
- Sorted frames in correct temporal order

Temporal Data Understanding

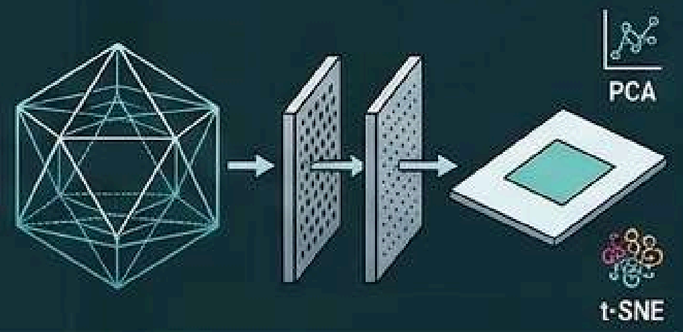
- Used consecutive frame pairs (t-5, t)
- Captured motion between frames
- Applied sliding window averaging for stability

FEATURE ENGNEERING

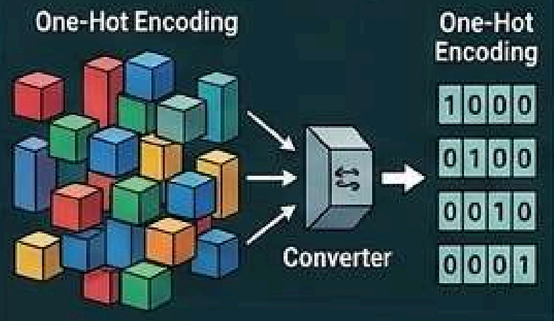
Handling Missing Values



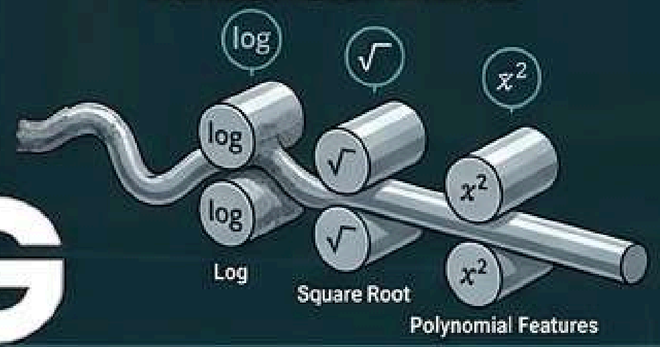
Dimensionality Reduction



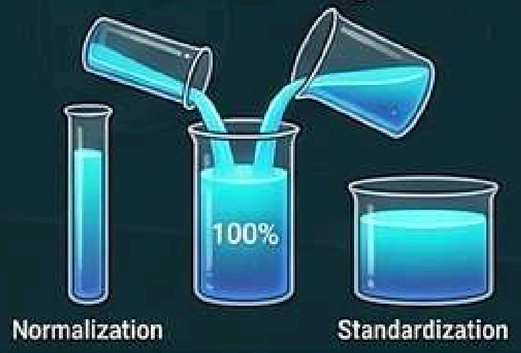
Encoding Techniques



Transformation Methods

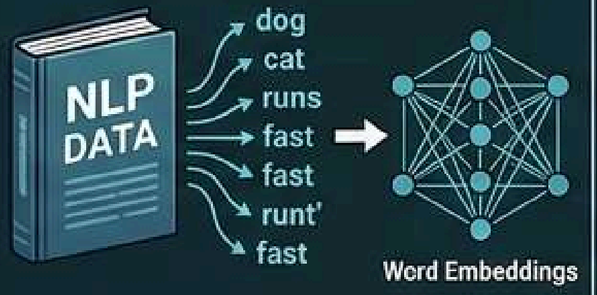


Feature Scaling



Advanced Engineering

Text Vectorization



Time Series

What is Feature Engneering ?

- Feature engineering is the process of transforming raw image data into meaningful numerical representations
- Since machine learning models cannot directly interpret pixel grids, we extract measurable properties such as:

1. Edges
2. Textures
3. Colors
4. Shapes

- It plays a critical role in improving model accuracy, robustness, and interpretability
- In essence, it bridges the gap between raw images and machine learning models

Motion Intensity

Total Motion Intensity (Mean Frame Difference)

- Measures the overall amount of change between consecutive frames
- Higher values indicate more movement in the scene
- Helps distinguish static vs dynamic regions

Direction Based Features

- Left vs Right Motion Difference
- Compares motion intensity between left and right halves of the frame
- Indicates horizontal movement direction
- Top vs Bottom Motion Difference
- Compares motion between upper and lower halves
- Captures vertical movement trends

```
# 2 & 3. Direction features (not raw left/right/top/bottom separately)
left_diff    = np.mean(np.abs(curr[:, :w//2] - prev[:, :w//2]))
right_diff   = np.mean(np.abs(curr[:, w//2:] - prev[:, w//2:]))
top_diff     = np.mean(np.abs(curr[:h//2, :] - prev[:h//2, :]))
bottom_diff  = np.mean(np.abs(curr[h//2:, :] - prev[h//2:, :]))
direction_lr = left_diff  - right_diff   # +ve = more motion on left
direction_tb = top_diff   - bottom_diff  # +ve = more motion on top
```

Optical Flow Features

Mean Horizontal Flow (Flow X)

- Represents average movement in the horizontal direction
- Positive → rightward motion, Negative → leftward motion

Mean Vertical Flow (Flow Y)

- Represents average vertical movement
- Positive → downward, Negative → upward

Overall Flow Magnitude

- Measures total strength of motion regardless of direction
- Captures how fast objects are moving

```
# Optical Flow
flow   = cv2.calcOpticalFlowFarneback(
            prev.astype(np.float32), curr.astype(np.float32),
            None, 0.5, 3, 15, 3, 5, 1.2, 0)
flow_x = flow[..., 0]
flow_y = flow[..., 1]
mag    = np.sqrt(flow_x**2 + flow_y**2)
```

Gradient-Based Features

- Sobel Gradients (Previous & Current Frames)
- Detect edges and intensity changes in x and y directions
- Capture structural changes between frames
- Useful for identifying motion boundaries and transitions

```
# 13. Sobel angle change between frames
gx_p        = cv2.Sobel(prev, cv2.CV_64F, 1, 0, ksize=3)
gy_p        = cv2.Sobel(prev, cv2.CV_64F, 0, 1, ksize=3)
gx_c        = cv2.Sobel(curr, cv2.CV_64F, 1, 0, ksize=3)
gy_c        = cv2.Sobel(curr, cv2.CV_64F, 0, 1, ksize=3)
ang_diff    = np.mean(np.abs(
                np.arctan2(gy_c, gx_c) - np.arctan2(gy_p, gx_p)))
```

Spatial Asymmetry Features

Left / Right / Top / Bottom Magnitudes

- Measure motion strength in different regions of the frame
- Helps detect localized movement patterns

Left–Right Difference

- Indicates turning or sideways motion

Top–Bottom Difference

- Indicates forward/backward or vertical motion bias

```
# 7 & 8. Quadrant asymmetry (left vs right, top vs bottom)
left_mag    = np.mean(mag[:, :w//2])
right_mag   = np.mean(mag[:, w//2:])
top_mag     = np.mean(mag[:h//2, :])
bottom_mag  = np.mean(mag[h//2:, :])
lr_mag_diff = left_mag   - right_mag   # turn indicator
tb_mag_diff = top_mag    - bottom_mag  # fwd/bwd indicator
```

Motion Dynamics Features

Flow Curl (Rotation)

- Measures rotational motion in the scene
- Higher values indicate turning or circular movement

Horizontal vs Vertical Gradient Ratio

- Compares horizontal and vertical changes in motion
- Helps identify dominant motion orientation

```
# 9. Flow curl (rotation) - sharp turns have high curl
curl        = np.mean(np.gradient(flow_x, axis=0) -
                      np.gradient(flow_y, axis=1))

# 10. HV gradient ratio
hv_ratio    = (np.mean(np.abs(np.gradient(diff_img, axis=1))) -
               np.mean(np.abs(np.gradient(diff_img, axis=0))))
```

Statistical Features

Variance of Frame Difference

- Measures how spread out motion intensity is
- High variance → uneven or complex motion

Entropy of Frame Difference

- Measures randomness or unpredictability in motion
- High entropy → chaotic movement

```
# 11. Variance of difference image
variance    = np.var(diff_img)

# 12. Entropy of difference image
hist, _     = np.histogram(diff_img, bins=20, range=(0,1), density=True)
hist       += 1e-6
entropy     = -np.sum(hist * np.log(hist))
```

Feature Scaling (Standardization)

Why Standardization?

- Features extracted have different scales and ranges
- Large-value features can dominate smaller ones
- Standardization ensures fair contribution from all features

Method Used

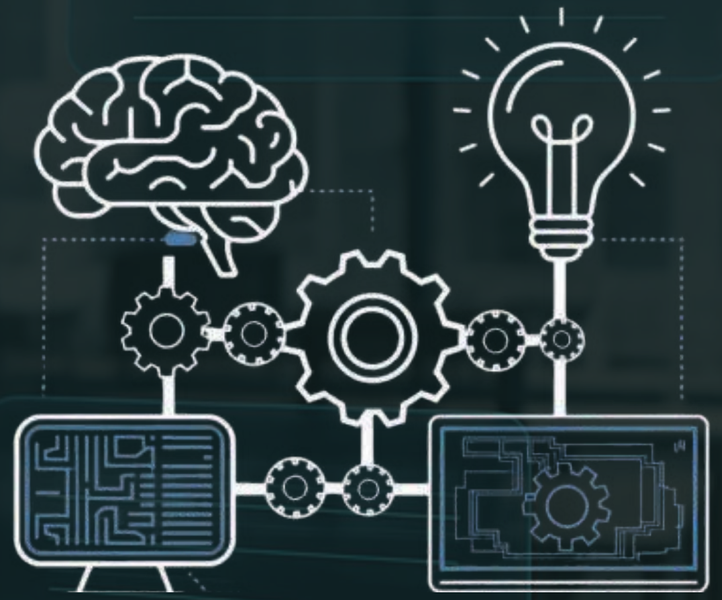
- Applied StandardScaler to normalize features
- Transforms data to:
- Mean = 0
- Standard Deviation = 1

```
scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)  # fit+transform on train
X_test  = scaler.transform(X_test_raw)       # only transform on test
```

Implementation

- Fitted scaler on training data
- Applied same transformation to test data
- Prevented data leakage

CHOICE OF MODEL



Choice of Model:

BASELINE

Always predicts majority class

- No learning involved.
- Accuracy = Class frequency
- Sanity-check reference

LOGISTIC REGRESSION

Linear classifier with class_weights = 'balanced'

- Fast and interpretable.
- Struggles with non-linear patterns.
- Underfit on complex features.

RANDOM FOREST

100 weights , class_weights = 'balanced'

- Non-linear interactions
- High Precision , Low Recall
- Threshold tuning limited

Hist Gradient Boosting

Final choice

- Handles imbalance natively
- Best Fi-macro after grid search
- Tuned: lr = 0.05 , depth = 5

Training Performance Comparison

Model	AHEAD F1	HALTED F1	SHARP-LEFT F1	SHARP-RIGHT F1	Macro F1
Logistic Regression	0.94	0.91	0.39	0.67	0.73
SVM	0.95	0.91	0.25	0.57	0.67
Random Forest*	0.96	0.92	0.31	0.63	0.71
Gradient Boosting*	0.96	0.89	0.53	0.62	0.75
Shallow ANN	0.95	0.84	0.48	0.41	0.67

- Gradient Boosting performs best overall (highest Macro F1)
- AHEAD is consistently well-predicted across all models
- SHARP-LEFT & SHARP-RIGHT are the most difficult classes
- Threshold tuning improves performance of tree-based models

Final Model Performance (Test Data) : Gradient Boosting

Class	Precision	Recall	F1-score
AHEAD	0.97	0.97	0.97
HALTED	0.95	0.84	0.89
SHARP-LEFT	0.63	0.67	0.65
SHARP-RIGHT	0.65	0.69	0.67

Achieved high accuracy with balanced performance across all motion classes

- The model generalizes well and does not overfit, which is why test performance is slightly better
- Strong performance on AHEAD & HALTED classes
- Significant improvement in SHARP turns detection
- Macro F1 indicates good balance despite class imbalance

Why Hist Gradient Boosting ??

- Handles class imbalance natively : class_weight='balanced' adjusts the decision boundary toward minority classes (HALTED, SHARP-LEFT, SHARP-RIGHT).
- Non - linear feature interactions : Boosted trees capture complex texture + color + edge interactions that Logistic Regression cannot model.
- Robust with SMOTE-Balanced Data : Gradient boosting generalizes well on synthetically oversampled training sets without overfitting.
- Grid search optimised(F1-macro) : 5-fold GridSearchCV over learning_rate, max_depth, max_iter, min_samples_leaf selected the best combination.

Threshold Optimization for Improved Classification

Problem

- Default threshold (0.5) led to poor performance on minority classes
- Model biased towards majority class (AHEAD)
- Result: Lower recall and F1-score for rare classes

Solution: Class-wise Threshold Tuning

- Introduced custom decision thresholds for each class
- Adjusted predicted probabilities before final classification
- Allowed flexible control over class predictions

Methodology

- Performed Grid Search over multiple threshold combinations
- Used Stratified 3-Fold Cross Validation
- Optimized for Macro F1-score to ensure balanced performance

Model Comparison

Model	Handle Imbalance	Non-linear	Inference Cost	Notes
Baseline (all 0)	—	—	—	Sanity Reference
Logistic Regression	Manual	✗	Low	Underfit
SVM (RBF)	Manual	✓	High	Slow training
Random Forest	Manual	✓	Low	Low recall
HistGradientBoosting	**✓ Native**	**✓**	**Low**	**CHOSEN ✓**

IMBALANCE HANDLING STRATERGIES

01

Undersample majority

AHEAD class (dominant) reduced to 2,000 samples using RandomUnderSampler to prevent the model from overwhelmingly predicting it.

02

Smote Minorities

HALTED, SHARP-LEFT, SHARP-RIGHT each upsampled to 1,500 using SMOTE (k=3 neighbors) on the training set only. Test data untouched.

03

class_weight='balanced'

HistGradientBoosting trained with balanced class weights. Shifts decision boundary toward minority classes during boosting iterations.

Precision-Recall Curves and ROC Curves

WHY PR CURVES ?

What are PR Curves?

Precision-Recall curves plot Precision (how many predicted positives are correct) vs Recall (how many actual positives were caught) across all decision thresholds. Area Under the Curve (AUC-PR) summarizes overall performance.

Why PR over ROC for this problem?

Our dataset is class-imbalanced — AHEAD dominates while SHARP-LEFT and SHARP-RIGHT are rare. ROC curves can be overly optimistic under imbalance. PR curves expose how well a model actually handles the minority classes.

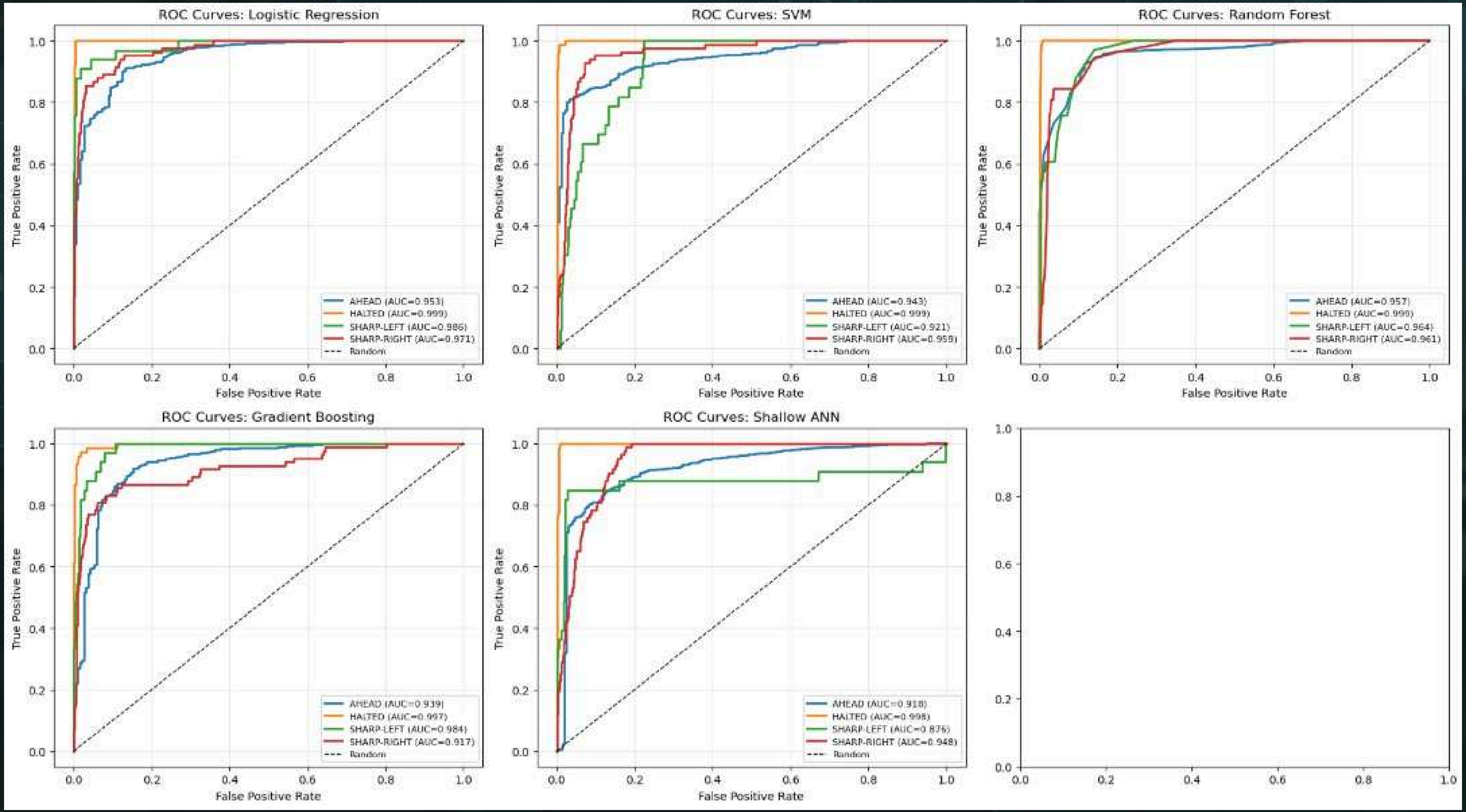
What a good curve looks like?

High AUC-PR = model maintains high precision even as recall increases. A curve hugging the top-right corner is ideal. A flat, low curve signals poor minority-class discrimination.

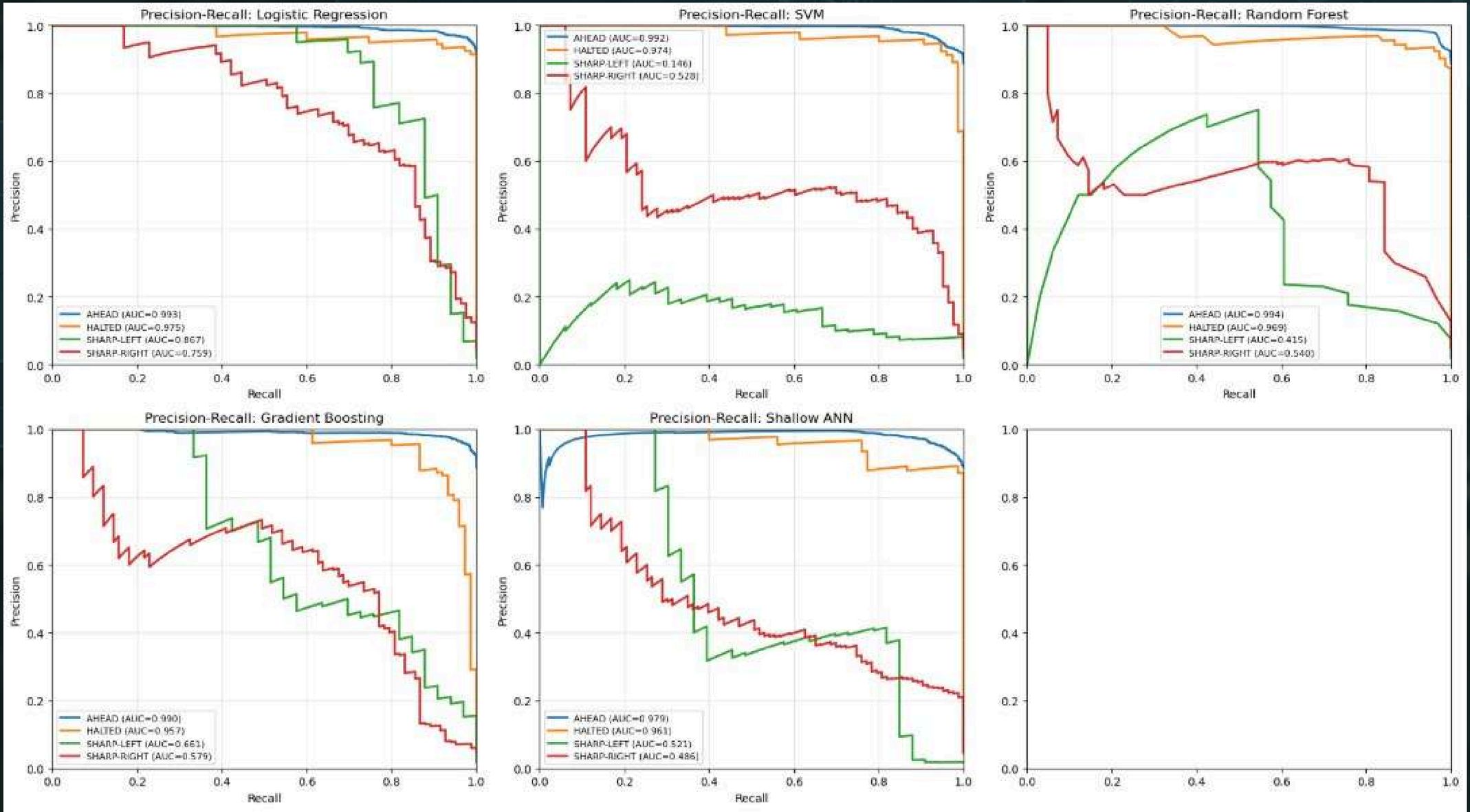
Key Insights from our data ?

SVM achieves ROC = 0.921 on Sharp-LEFT but PR AUC = 0.146 – a near total collapse. This shows PR curves are the more revealing metric for this imbalanced multi-class problem.

ROC CURVES-ALL MODELS


ROC Curves: Logistic Regression
AHEAD (AUC=0.953)
HALTED (AUC=0.999)
SHARP-LEFT (AUC=0.986)
SHARP-RIGHT (AUC=0.971)
Random
ROC Curves: SVM
AHEAD (AUC=0.943)
HALTED (AUC=0.999)
SHARP-LEFT (AUC=0.921)
SHARP-RIGHT (AUC=0.959)
Random
ROC Curves: Random Forest
AHEAD (AUC=0.957)
HALTED (AUC=0.999)
SHARP-LEFT (AUC=0.964)
SHARP-RIGHT (AUC=0.961)
Random
ROC Curves: Gradient Boosting
AHEAD (AUC=0.939)
HALTED (AUC=0.997)
SHARP-LEFT (AUC=0.984)
SHARP-RIGHT (AUC=0.917)
Random
ROC Curves: Shallow ANN
AHEAD (AUC=0.918)
HALTED (AUC=0.998)
SHARP-LEFT (AUC=0.876)
SHARP-RIGHT (AUC=0.948)
Random
True Positive Rate
False Positive Rate

PRECISION-RECALL CURVES-ALL MODELS

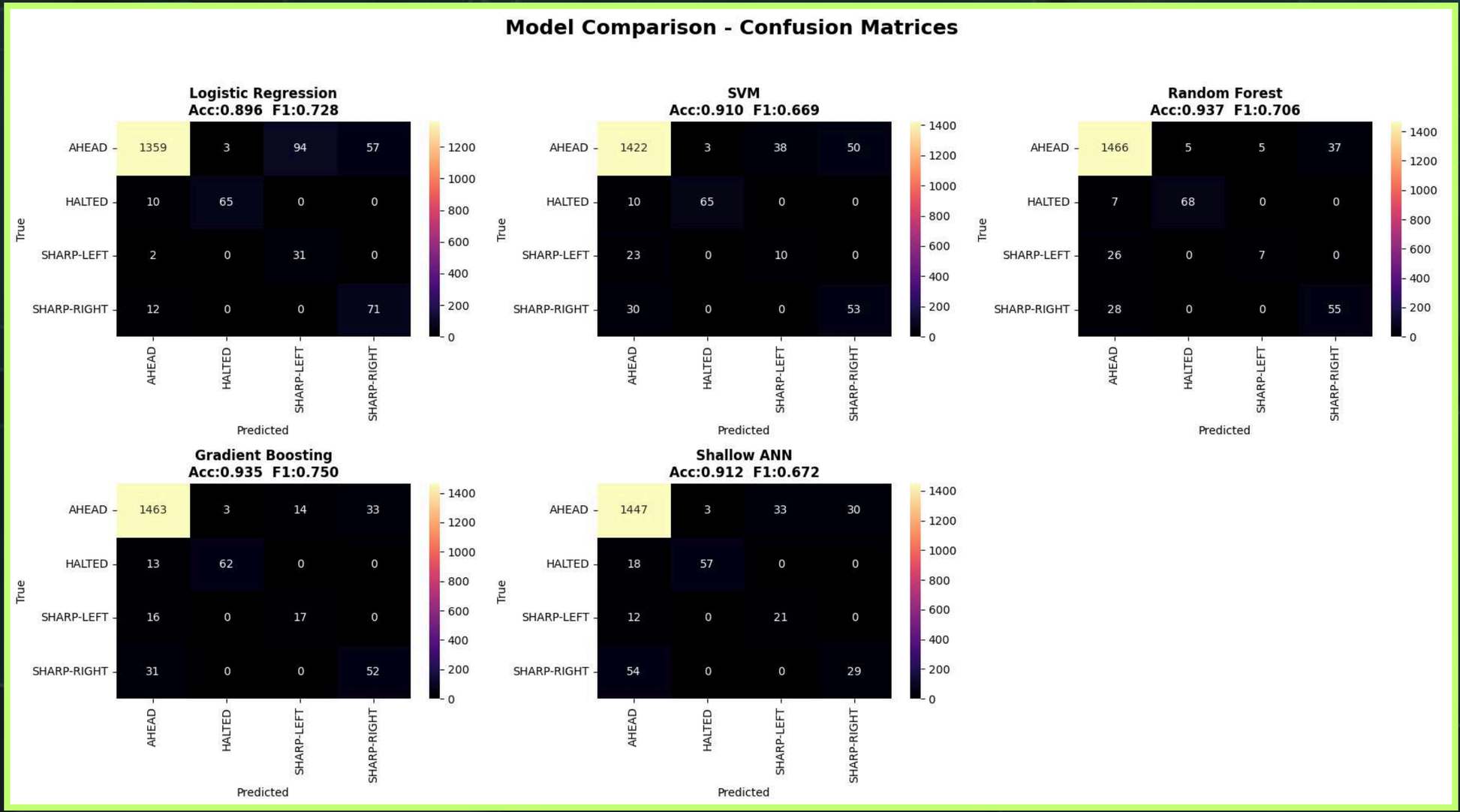

Precision-Recall: Logistic Regression
AHEAD (AUC=0.993)
HALTED (AUC=0.975)
SHARP-LEFT (AUC=0.867)
SHARP-RIGHT (AUC=0.759)
Precision-Recall: SVM
AHEAD (AUC=0.992)
HALTED (AUC=0.974)
SHARP-LEFT (AUC=0.146)
SHARP-RIGHT (AUC=0.528)
Precision-Recall: Random Forest
AHEAD (AUC=0.994)
HALTED (AUC=0.969)
SHARP-LEFT (AUC=0.415)
SHARP-RIGHT (AUC=0.540)
Precision-Recall: Gradient Boosting
AHEAD (AUC=0.990)
HALTED (AUC=0.957)
SHARP-LEFT (AUC=0.661)
SHARP-RIGHT (AUC=0.579)
Precision-Recall: Shallow ANN
AHEAD (AUC=0.979)
HALTED (AUC=0.961)
SHARP-LEFT (AUC=0.521)
SHARP-RIGHT (AUC=0.486)
Precision
Recall

Model Performance on Training Data

Model	AHEAD AUC	HALTED AUC	SHARP-LEFT AUC	SHARP-RIGHT AUC
Logistic Regression	0.953	0.999	0.886	0.971
SVM	0.943	0.999	0.921	0.959
Random Forest	0.957	0.999	0.964	0.961
Gradient Boosting	0.939	0.997	0.984	0.917
Shallow ANN	0.918	0.998	0.876	0.948

Key observation: All models achieve near-perfect AUC on HALTED (≥0.997). SHARP-LEFT and SHARP-RIGHT are the hardest classes — Gradient Boosting leads on SHARP-LEFT (0.984) while Random Forest is strong overall.

CONFUSION MATRIX:



Hurdles

- Handling 6GB of Raw Data: Processing large volumes of sequential images was highly resource-intensive, requiring continuous code optimization to keep training cycles efficient.
- Manual Data Labeling: Annotating thousands of frames by hand required strict consistency rules to ensure our ground truth data remained accurate and reliable.
- Preventing Data Leakage: Because frames are sequential, standard random splits would let the model "cheat." We resolved this by strictly splitting data by sequence boundaries.
- Severe Class Imbalance: Straight-driving frames vastly outnumbered turns. We had to carefully stratify our splits to ensure rare turning events were represented fairly.
- Optimizing Meaningful Metrics: Early models achieved high accuracy simply by always guessing "AHEAD." We fixed this by adjusting class weights and thresholds, focusing entirely on the F1-Score.

Experimentation On Feature Engineering

- First, we used a tool called FFT to separate smooth, straight driving from sudden, jerky movements. The catch is, FFT is great at spotting what happened, but it loses track of when it happened.
- To solve this, we brought in Haar Wavelets You can think of this as our mathematical alarm system—it triggers the exact moment a vehicle snaps into a sharp turn
- Finally, we had to be careful with how we graded the model. Since vehicles in our data are driving straight most of the time, the model could basically cheat and get a high score just by guessing 'straight' every single time
- To prevent this, we used a metric called a Weighted F1-Score, which strictly penalizes the model if it misses those rare, critical turns

Experimentation On Data Augmentation

- Our dataset had a serious class imbalance as many of them are ahead and only few are SHARP-LEFT and SHARP-RIGHT.

- To fix this without collecting new data, we horizontally flipped existing LEFT turn images to synthetically generate RIGHT turn equivalents, and vice versa. A mirrored left turn is physically identical to a right turn, making this augmentation mathematically valid, not just a shortcut.

- This effectively doubled our minority class samples, giving the model balanced exposure to both directions during training. The result our model stopped being biased toward one side and learned to recognize turns in both directions equally well.

Failures

- Experiment 1: Fourier Transforms: We added Fourier Transforms hoping to capture complex motion signals. However, it overcomplicated the model and actually reduced our F1-Score compared to using our simpler baseline features.

- Experiment 2: Data Augmentation: To handle the massive class imbalance, we tried standard image-based data augmentation. Unfortunately, it failed to capture true sequential motion, resulting in a lower F1-Score than our mathematical balancing approach (SMOTE).

- The Core Learning: For sequential motion, adding highly complex features or standard image tricks isn't always the answer. Simpler, targeted logic proved much more effective.

Key Takeaways:

- 1. Accuracy can be misleading

One of the biggest surprises for us was realizing that a 90% accuracy doesn't always mean the model is good. In our case, most of the data was just vehicles going straight, so the model kept predicting "AHEAD" and still got high accuracy. But it was actually failing to detect turns. That's when we understood that accuracy alone isn't enough. Metrics like F1-score and recall gave us a much clearer picture of what was really happening.

- 2. Good data matters more than complex models

We spent a lot of time fixing our labels and organizing the data properly, sometimes more than actually working on the models. At first, it felt like we were wasting time, but later we realized this was the most important part. Even a simple model works really well if the data is clean and structured. On the other hand, even the best algorithms fail if the data is messy.

- 3. Sequential data should be treated carefully

Initially, we handled video frames like random images and shuffled them freely. But later we realized that frames are connected — one frame depends on the previous one. When we stopped shuffling randomly and instead split the data based on sequences, the model stopped "cheating" and the results became much more realistic and reliable.

Key Takeaways:

- 4. Math actually helped us a lot

Concepts like Fourier Transforms and Wavelets felt very theoretical at first. But when we applied them, things started to make sense. For example, the Haar wavelet was able to catch sudden changes like sharp turns very clearly, which our basic methods missed. That was a big moment for us where theory actually connected with real-world results.

- 5. Class imbalance is really a data issue

We tried many techniques like SMOTE, undersampling, and tuning thresholds to handle class imbalance. They helped, but it always felt like a temporary fix. What we learned is that the real solution is to collect balanced data from the beginning. Fixing it later in code works, but it’s not ideal.

- 6. The toughest part wasn’t coding

Honestly, coding was not the hardest part of this project. The difficult part was making decisions while labeling the data. For example, deciding when a slight movement becomes a “sharp turn” was not easy. These are human judgment calls that no algorithm can make for you. It required patience, consistency, and really understanding our data.

THANK YOU